Practical 2: State Management and Routing in React

Objective	**To implement reactive state management using useState and multi-page navigation using React Router in the existing portfolio application.**
Problem Statement	**• Extend the portfolio application from Practical 1 by adding React Router.** **• Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.** **• Each route must render a distinct component without a full page reload.** **• Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.** **• The Contact page must include a controlled input that captures and displays user input in real time.**
Project Structure	App.jsx (wrapped in BrowserRouter) ├── NavBar.jsx (links to all routes) ├── Route: → Home.jsx ├── Route: → Projects.jsx └── Route: → Contact.jsx (controlled form,useState)
Implementation	1. Install React Router in the existing portfolio project: npm install react-router-dom 2. Wrap the App component with BrowserRouter in main.jsx: import { BrowserRouter } from 'react-router-dom'; <BrowserRouter> <App /> </BrowserRouter> 3. Define 3 routes inside App.jsx using Routes and Route: <Routes> <Route path="/" element={<Home />} /> <Route path="/projects" element={<Projects />} /> <Route path="/contact" element={<Contact />} /> </Routes> 4. Build the NavBar using Link (not <a> tags) to avoid full page reloads. 5. Inside Contact.jsx, create a controlled input tied to useState: const [message, setMessage] = useState(''); <input value={message} onChange={(e) => setMessage(e.target.value)} /> 6. Add a second useState variable to toggle visibility of an element (e.g., a help tooltip). 7. Test navigation across all 3 routes and confirm no page reload occurs.

Code

Wrap App with BrowserRouter

```
const rootElement = document.getElementById('root')

ReactDOM.createRoot(rootElement).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
)
```

Contact.jsx:

```
import React from 'react'

function Contact() {
  const emailPlaceholder = 'jemitvaghasiya07@gmail.com'
  const linkedinPlaceholder = 'https://www.linkedin.com/in/jemitvaghasiya/'
  const messagePlaceholder = 'https://jemitportfolio.netlify.app/contact'

  const openLink = (url) => {
    window.open(url, '_blank', 'noopener,noreferrer')
  }

  return (
    <div className="contact-info">
      <h3>Contact Me</h3>

      <div className="contact-buttons">
        <button
          type="button"
          className="btn email-btn"
          onClick={() => openLink(`mailto:${emailPlaceholder}`)}
        >
          Email me
        </button>

        <button
          type="button"
          className="btn linkedin-btn"
          onClick={() => openLink(linkedinPlaceholder)}
        >
          LinkedIn
        </button>

        <button
          type="button"
          className="btn message-btn"
          onClick={() => openLink(messagePlaceholder)}
        >
```

```
          Message me
        </button>
      </div>
    </div>
  )
}

export default Contact
```

Skill.jsx

```
function Skills({ skills }) {
  return (
    <section>
      <h2>Skills</h2>
    <ul>
      {skills.map((skill, index) => (
        <li key={index} className="skill-badge">{skill}</li>
      ))}
    </ul>
    </section>
  )
}

export default Skills
```

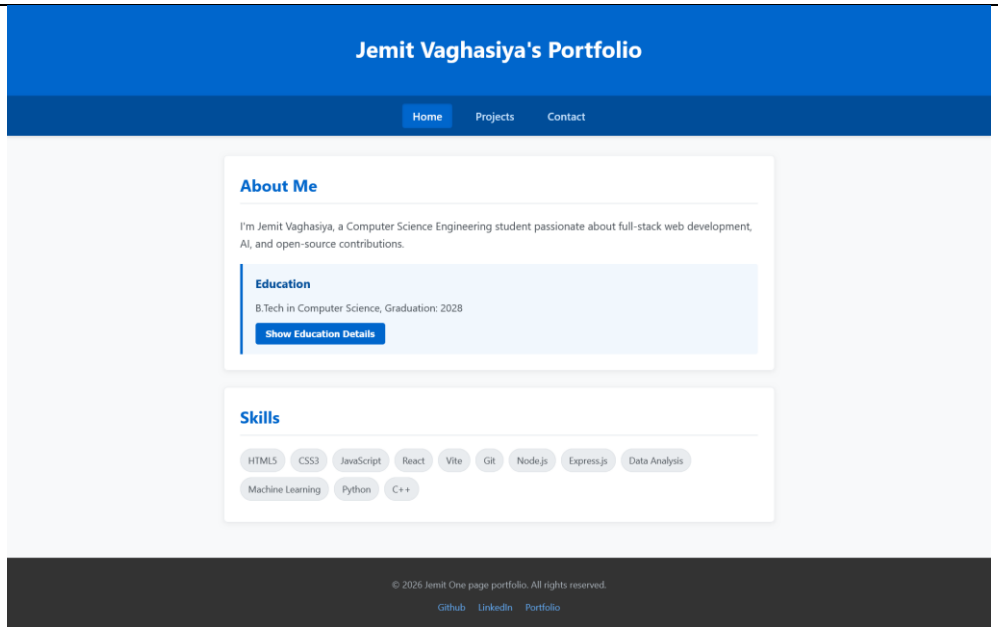
Project.jsx

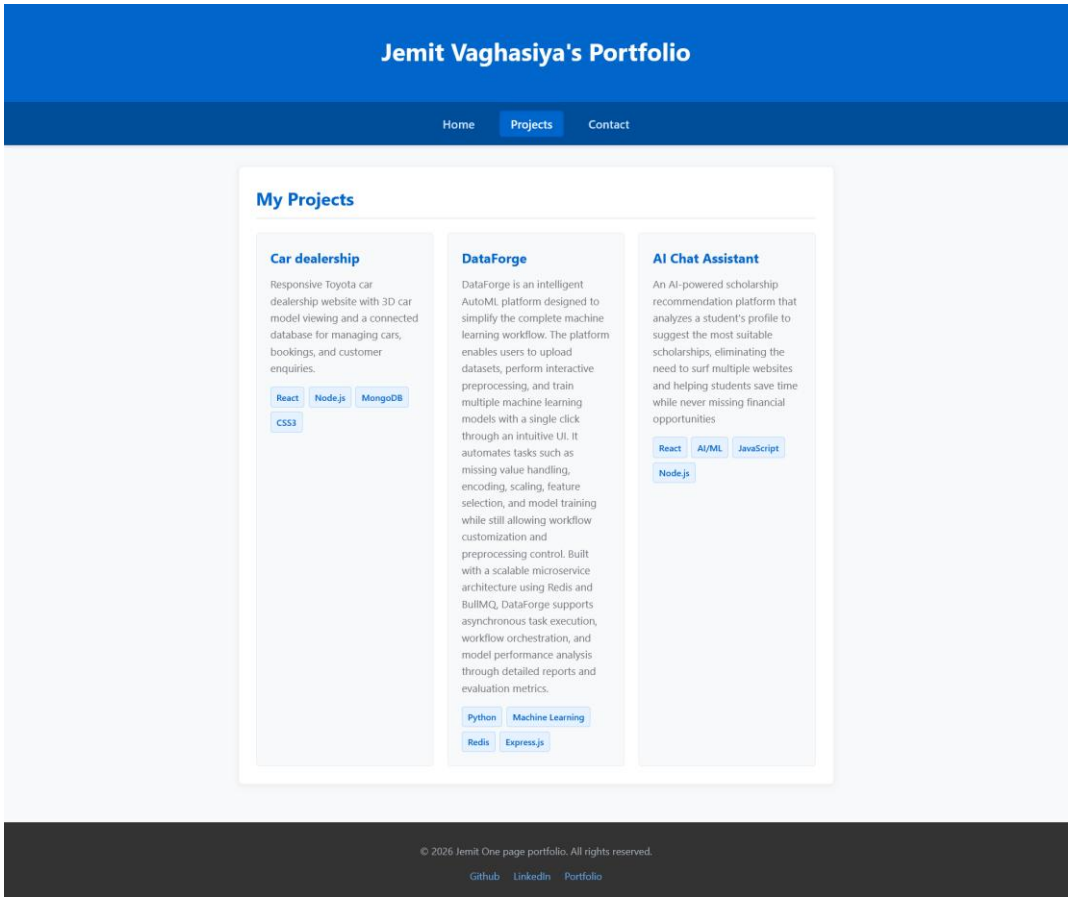
```
function Projects() {
  const [repos, setRepos] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [searchQuery, setSearchQuery] = useState('')

  useEffect(() => {
    const fetchRepos = async () => {
      try {
        setLoading(true)
        setError(null)

        // Fetch up to 100 repositories sorted by most recently updated
        const response = await fetch(
          `https://api.github.com/users/${GITHUB_USERNAME}/repos?sort=updated&per_page=100`
        )

        if (!response.ok) {
          if (response.status === 404) {
```

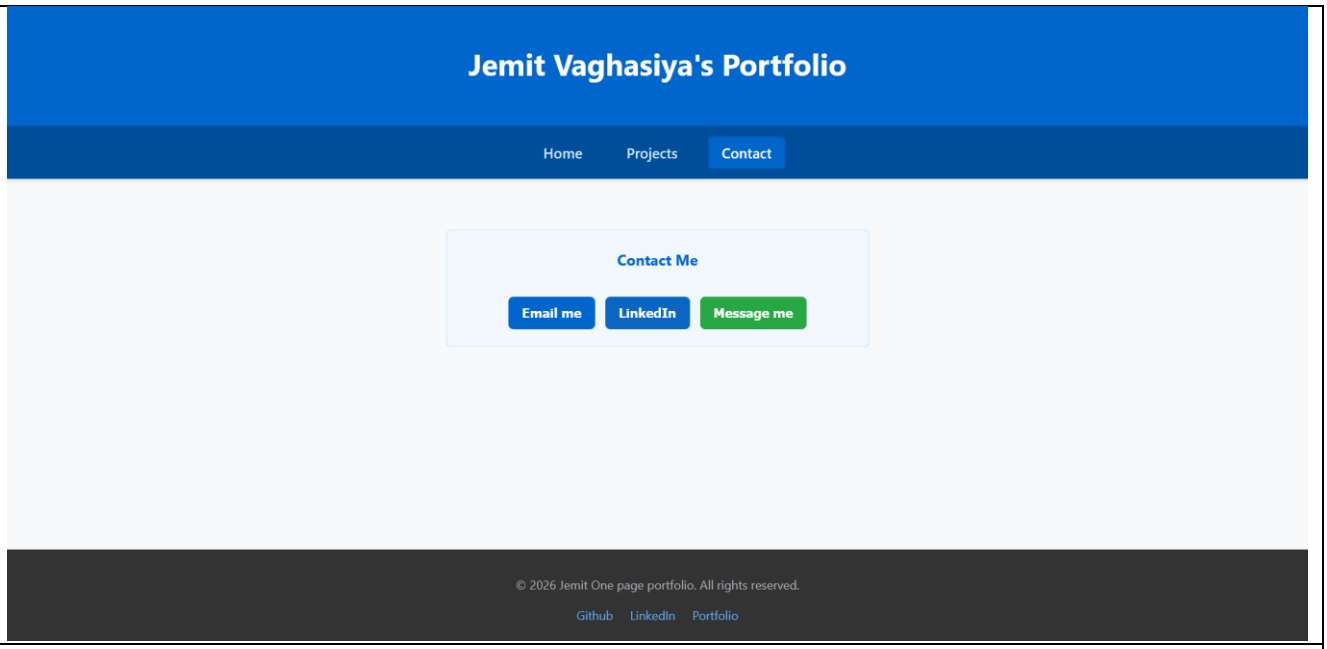
```
          throw new Error(`GitHub user "${GITHUB_USERNAME}" not found.`)
        } else if (response.status === 403) {
          throw new Error('API rate limit exceeded. Please try again later.')
        } else {
          throw new Error(`Failed to fetch repositories (Status ${response.status}).`)
        }
      }

      const data = await response.json()
      // Filter out forks if you only want source repos, or keep them all. Let's keep source repos first
or all.
      // Let's show all repositories but sort them.
      setRepos(data)
    } catch (err) {
      setError(err.message || 'An error occurred while fetching repositories.')
    } finally {
      setLoading(false)
    }
  }

  fetchRepos()
 }, [])

 // Filter repositories based on search query (name, description, or language)
 const filteredRepos = repos.filter((repo) => {
  const query = searchQuery.toLowerCase()
  const nameMatch = repo.name?.toLowerCase().includes(query)
  const descMatch = repo.description?.toLowerCase().includes(query)
  const langMatch = repo.language?.toLowerCase().includes(query)
  return nameMatch || descMatch || langMatch
 })

 return (
  <div className="projects-container">
   <h2>My Projects</h2>

   {/* Search Bar */}
   <div className="search-container">
    <input
     type="text"
     placeholder="Search repositories by name, description, or language..."
     value={searchQuery}
     onChange={(e) => setSearchQuery(e.target.value)}
     className="search-input"
    />
    {searchQuery && (
     <button
      onClick={() => setSearchQuery('')}
      className="search-clear-btn"
      title="Clear search"
```

```
        >
          &times;
        </button>
      )}
    </div>
```

Output





	
Conclusion	we learned how to use React Router to move between different pages without reloading the website. We also used useState to manage user input and show or hide content. This made the portfolio project more interactive, easy to use, and well organized.